

# Lecture 25: Mixture of experts and efficiency

PSYC 51.17: Models of language and communication

Jeremy R. Manning  
Dartmouth College  
Winter 2026

# Learning objectives

By the end of this lecture, you will be able to...

- Explain the **scaling dilemma** and why dense models are computationally wasteful
- Describe the **Mixture of Experts (MoE)** architecture: experts, routers, and sparse activation
- Implement a **simplified MoE layer** in PyTorch
- Discuss **load balancing** challenges and their solutions
- Evaluate efficiency techniques: **quantization, distillation, Flash Attention**, and **speculative decoding**
- Assess the role of efficiency in **democratizing access** to powerful AI

# The scaling dilemma

Bigger models perform better, but at enormous cost

| Model | Parameters   | Estimated training cost | CO <sub>2</sub> equivalent |
|-------|--------------|-------------------------|----------------------------|
| GPT-1 | 117M         | ~\$10K                  | Negligible                 |
| GPT-3 | 175B         | ~\$4.6M                 | 112 cars for 1 year        |
| GPT-4 | ~1.8T (est.) | ~\$100M+                | Unknown                    |

## The key insight

In a dense model like GPT-3, **every parameter is active for every input token**. But not all parameters need to be active for every input. A math question does not require the same circuitry as a French translation. **Can we activate only the relevant parameters?**

# Dense vs. sparse models

## Two approaches to scaling

- **Dense models** (GPT-3, Llama): All parameters are active for every token. 175B parameters = 175B active.
- **Sparse models** (MoE): Only a subset of parameters are active per token. 47B total parameters, but only 13B active per token (Mixtral).

## The MoE tradeoff

| Dimension                   | Dense    | Sparse (MoE)      |
|-----------------------------|----------|-------------------|
| Active parameters per token | 100%     | ~25%              |
| Total parameters (memory)   | N        | 3--8x N           |
| Compute per token           | High     | Low               |
| Architecture complexity     | Simple   | Complex (routing) |
| Quality at matched compute  | Baseline | <b>Higher</b>     |

MoE gives you more *capacity* (knowledge storage) with less *compute* (inference cost).

# What is Mixture of Experts?

Shazeer et al. (2017)

A **Mixture of Experts** layer replaces the single feed-forward network in each transformer block with **multiple parallel feed-forward networks** (experts) and a **router** that selects which experts process each token.

Key components:

1. **Experts**: N independent feed-forward networks (typically 8--64)
2. **Router (gate)**: A learned linear layer that assigns tokens to experts
3. **Top-k selection**: Only the top-k experts (typically  $k=2$ ) are activated per token

## Intuition

Think of experts as specialists on a team. When a code-related token arrives, the "code expert" handles it. When a French token arrives, the "French expert" activates. The router is the dispatcher.

# The router mechanism

## Routing a token to top-2 experts

```
1 import torch.nn.functional as F
2
3 def route_token(x, router_weights, num_experts=8, k=2):
4     # Step 1: Compute routing scores
5     logits = x @ router_weights          # (num_experts,)
6     # e.g., [-0.5, 2.1, 1.3, 0.2, -0.1, 0.0, -0.3, 0.1]
7
8     # Step 2: Convert to probabilities
9     probs = F.softmax(logits, dim=-1)
10    # e.g., [0.04, 0.52, 0.24, 0.08, 0.03, 0.03, 0.03, 0.03]
11
12    # Step 3: Select top-k experts
13    top_k_probs, top_k_indices = torch.topk(probs, k)
14    # indices: [1, 2], probs: [0.52, 0.24]
```

continued...



# The router mechanism

## Routing a token to top-2 experts

```
15  
16     # Step 4: Normalize selected weights  
17     top_k_probs = top_k_probs / top_k_probs.sum() # [0.68, 0.32]  
18     return top_k_indices, top_k_probs
```

...continued

Output: Expert 1 handles 68%, Expert 2 handles 32%.  
The other 6 experts do **no computation**.

# MoE layer in PyTorch

## Simplified implementation

```
1 class MoELayer(nn.Module):  
2     def __init__(self, d_model, num_experts=8, k=2):  
3         super().__init__()  
4         self.k = k  
5         self.gate = nn.Linear(d_model, num_experts)  
6         self.experts = nn.ModuleList([  
7             nn.Sequential(  
8                 nn.Linear(d_model, 4 * d_model),  
9                 nn.GELU(),
```

continued...



# MoE layer in PyTorch

## Simplified implementation

```
10         nn.Linear(4 * d_model, d_model)
11     ) for _ in range(num_experts)
12 ]
13
14     def forward(self, x):
15         router_probs = F.softmax(self.gate(x), dim=-1)
16         top_k_probs, top_k_idx = torch.topk(router_probs, self.k,
17         dim=-1)
18         top_k_probs = top_k_probs / top_k_probs.sum(dim=-1,
19         keepdim=True)
20         output = torch.zeros_like(x)
```

...continued...

# MoE layer in PyTorch

## Simplified implementation

```
19         for i in range(self.k):
20             for eid in range(len(self.experts)):
21                 mask = (top_k_idx[ ..., i] == eid)
22                 if mask.any():
23                     output[mask] += top_k_probs[ ..., i]
24                     [mask].unsqueeze(-1) \
25                                     * self.experts[eid](x[mask])
25         return output
```

...continued

# Load balancing

## The routing collapse problem

Without intervention, the router tends to send most tokens to a few "popular" experts, leaving others undertrained or completely **dead** (never activated). This wastes parameters and reduces model capacity.

## Two solutions

**1. Auxiliary load-balancing loss** (added to training objective):

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$$

where  $f_i$  is the fraction of tokens routed to expert  $i$ ,  $P_i$  is the average routing probability for expert  $i$ , and  $\alpha$  is a small coefficient ( $\sim 0.01$ ). This penalizes imbalanced routing.

**2. Expert capacity limits:** Each expert has a maximum number of tokens it can process per batch. Overflow tokens are routed to the next-best expert.

# Training challenges

## What makes MoE training harder than dense models

| Challenge                      | Cause                                           | Mitigation                                |
|--------------------------------|-------------------------------------------------|-------------------------------------------|
| <b>Routing collapse</b>        | Router converges to always picking same experts | Load-balancing loss, random routing       |
| <b>Dead experts</b>            | Some experts never receive tokens               | Expert dropout, periodic reinitialization |
| <b>High variance gradients</b> | Discrete routing decisions                      | Larger batch sizes, softmax gating        |
| <b>Memory overhead</b>         | All experts must be in memory                   | Expert parallelism across GPUs            |
| <b>Communication cost</b>      | Tokens must be sent to correct GPU              | Optimized all-to-all communication        |

## Questions to consider

Despite these challenges, MoE models are becoming the default for frontier models (Mixtral, GPT-4 reportedly uses MoE). What does this tell us about the compute-quality tradeoff?

# Mixtral 8x7B

Jiang et al. (2024): the MoE model that changed the game

**Mixtral** from Mistral AI demonstrated that open-weight MoE models can match or exceed much larger dense models:

| Specification               | Value                                     |
|-----------------------------|-------------------------------------------|
| Total parameters            | 47B (8 experts x 7B, minus shared layers) |
| Active parameters per token | 13B (top-2 routing)                       |
| Transformer layers          | 32                                        |
| Context window              | 32K tokens                                |
| License                     | Apache 2.0 (fully open)                   |

## The headline result

Mixtral (47B total, 13B active) **matches Llama 2 70B** on benchmarks while running at **5x the speed**. Quality of a 70B model at the cost of a 13B model.

# What do experts learn?

## Emergent specialization (no supervision required)

Analysis of Mixtral's routing patterns reveals natural specialization:

| Token type     | Primary expert | Example tokens                                                                           |
|----------------|----------------|------------------------------------------------------------------------------------------|
| Code           | Expert 2       | <code>def</code> , <code>class</code> , <code>import</code>                              |
| French         | Expert 5       | <code>la</code> , <code>le</code> , <code>français</code>                                |
| Math           | Expert 7       | <code><math>\Sigma</math></code> , <code><math>\int</math></code> , <code>theorem</code> |
| Common English | Expert 1       | <code>the</code> , <code>is</code> , <code>and</code>                                    |
| Technical      | Expert 3       | <code>neural</code> , <code>gradient</code>                                              |

## Questions to consider

Nobody told Expert 2 to specialize in code -- it emerged from training. This mirrors how brain regions develop functional specialization. What does this suggest about the relationship between architecture and learned structure?



# Model compression: quantization

Reducing precision to reduce memory and speed up inference

**Quantization** converts model weights from high-precision floating point to lower-precision integers, dramatically reducing memory and often improving speed.

Memory savings for a 7B parameter model

| Precision   | Bits per weight | Model size | Fits on...   |
|-------------|-----------------|------------|--------------|
| FP32        | 32              | 28 GB      | High-end GPU |
| FP16 / BF16 | 16              | 14 GB      | Consumer GPU |
| INT8        | 8               | 7 GB       | Gaming GPU   |
| INT4        | 4               | 3.5 GB     | Laptop GPU   |

INT4 quantization enables running a **7 billion parameter model on a laptop** with only 1--2% quality degradation on most benchmarks.

# Knowledge distillation

## Training a small model to mimic a large one

In **knowledge distillation**, a small "student" model is trained to match the output distribution of a large "teacher" model, rather than training from raw data alone. The student learns the teacher's "dark knowledge" -- the relative probabilities across all tokens, not just the top-1 prediction.

## Distillation in practice

```
1  # Teacher: Large model (frozen, no gradients)
2  teacher_logits = large_model(input) # (batch, seq, vocab)
3
4  # Student: Small model (being trained)
5  student_logits = small_model(input)
6
7  # Distillation loss: match teacher's soft probabilities
8  loss = F.kl_div(
9      F.log_softmax(student_logits / temperature, dim=-1),
10     F.softmax(teacher_logits / temperature, dim=-1))
```

# Knowledge distillation

## Distillation in practice

```
11     reduction='batchmean'  
12 )
```

...continued

**Result:** DistilBERT is 40% smaller than BERT but retains 97% of its performance.

# Inference optimizations

## Making generation faster without changing the model

**KV cache:** During generation, cache key and value tensors from previous tokens. Avoids recomputing attention over the entire sequence at each step. Reduces per-token cost from  $O(n)$  to  $O(1)$ .

**Flash Attention** (Dao et al., 2022): Tiled, memory-efficient attention that avoids materializing the full  $n \times n$  attention matrix. 2--4x faster and enables longer sequences.

**Speculative decoding:** Use a small "draft" model to propose several tokens at once, then verify them in parallel with the large model. If 4 out of 5 draft tokens are accepted, you get ~4x speedup with identical output quality.

**Continuous batching** (vLLM): New requests join an ongoing batch without waiting for the longest sequence to finish. Dramatically improves throughput for serving.

# State space models: an alternative to attention

Mamba (Gu & Dao, 2023)

**State space models (SSMs)** replace self-attention with a recurrent mechanism that processes sequences in **linear time**  $O(n)$  instead of quadratic  $O(n^2)$ :

```
1 state = initial_state          # Fixed size, independent of sequence
  length
2 for token in sequence:
3     state = A @ state + B @ token  # Update state
4     output = C @ state             # Read output
```

## Attention vs. SSM scaling

| Sequence length | Attention cost | SSM cost |
|-----------------|----------------|----------|
| 1K              | 1x             | 1x       |
| 4K              | 16x            | 4x       |
| 16K             | 256x           | 16x      |
| 64K             | 4,096x         | 64x      |

SSMs scale linearly, making them attractive for very long sequences. Hybrid architectures (attention + SSM)

# The efficiency landscape

## Choosing the right technique

| Technique            | Best for                  | Main tradeoff                    |
|----------------------|---------------------------|----------------------------------|
| Dense large model    | Maximum quality           | Expensive, slow                  |
| MoE                  | Quality + speed           | High memory (all experts stored) |
| Quantization         | Edge / local deployment   | Small quality loss               |
| Distillation         | Fixed tasks, small budget | Requires teacher model           |
| Flash Attention      | Longer contexts           | Implementation complexity        |
| Speculative decoding | Faster generation         | Requires draft model             |
| SSMs (Mamba)         | Very long sequences       | Less proven than attention       |

## Decision heuristic

Need max quality? → Dense large. Need quality + speed on GPU? → MoE. Need to run on a laptop? → Quantized small model. Need extremely long contexts? → Hybrid attention + SSM.



# Environmental impact and democratization

## The carbon cost of scale

Training GPT-3 produced an estimated **502 tonnes of CO<sub>2</sub>** -- equivalent to 112 cars driven for a year (Strubell et al., 2019). As models grow, so does their environmental footprint.

## Efficiency enables access

Efficiency techniques are not just about cost -- they are about **who gets to use AI**:

- **Quantized open models** (Llama, Mixtral) run on consumer hardware
- **LoRA / QLoRA** enable fine-tuning on a single GPU
- **Smaller efficient models** (Phi, Gemma) bring quality to resource-constrained settings
- **Open weights** let researchers, startups, and developing nations participate in AI development

Without efficiency research, frontier AI would remain exclusive to a handful of well-funded organizations.

# Key takeaways

## Core concepts from this lecture

1. **MoE** replaces dense feed-forward layers with multiple experts and a router, activating only top-k per token
2. **Mixtral 8x7B** achieves 70B-quality at 13B-cost, proving MoE works at scale
3. **Load balancing** (auxiliary loss, capacity limits) is critical to prevent routing collapse
4. **Quantization** (INT4/INT8) enables running large models on consumer hardware
5. **Knowledge distillation** transfers capability from large teacher to small student models
6. **Inference optimizations** (KV cache, Flash Attention, speculative decoding) dramatically speed up generation
7. **Efficiency is an equity issue**: it determines who can access and benefit from AI

# Further reading

## References

- **Shazeer et al. (2017)** -- "Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer" [[arXiv](#)]
- **Jiang et al. (2024)** -- "Mixtral of Experts" [[arXiv](#)]
- **Fedus et al. (2022)** -- "Switch Transformers" [[arXiv](#)]
- **Dao et al. (2022)** -- "FlashAttention" [[arXiv](#)]
- **Gu & Dao (2023)** -- "Mamba: Linear-Time Sequence Modeling" [[arXiv](#)]
- **Strubell et al. (2019)** -- "Energy and Policy Considerations for Deep Learning" [[arXiv](#)]

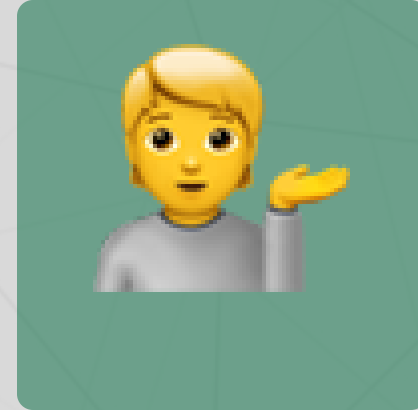
# Questions?



Email



Discord



Office hours

Up next...

Ethics, bias, and safety: responsible development of large language models